

ERDOS RESEARCH

POSITION PAPER

Using Claude Skills to Learn Coding for Non-Developers

A skills-as-substrate approach to coding literacy, in which domain experts learn to direct, read, and verify code through Claude — and build a reusable personal skills library along the way.

By [Ajit Jaokar](#)

Erdos Research

AI Research & Education | United Kingdom

May 2026

Contents

Contents	2
Abstract	3
1. What Are Claude Skills?	3
2. A Taxonomy of Skill Types for Teaching Coding	4
2.1 Markup and prompt skills as first-class skill types	4
Markup skills	4
Prompt skills	4
3. How the Skills Build on One Another	5
4. The Learning Loop	5
5. Significance of the Method	6
6. An End-to-End Example	6
Step 1 — The brief	6
Step 2 — Apply a prompt skill	7
Step 3 — Claude produces the code; the analyst reads it	7
Step 4 — Hit a problem, apply a diagnostic skill	7
Step 5 — Record it as a skill	7
7. Implementing the Method: A Teacher's Workflow	8
7.1 What the Teacher Creates in Advance	8
7.2 How Students Invoke the Skills	9
7.3 The Live Teaching Loop	9
7.4 The Student's Learning Loop	10
7.5 How It Adds Up Over a Course	10
8. Conclusion	11

Abstract

Non-developers do not need to become programmers. They need to reach the point where they can direct code, read code, and verify code — using Claude as the execution layer. This paper sets out a method for achieving that, built on the observation that a Claude Skill, expressed as a SKILL.md file, is an ideal vehicle for coding literacy. A skill packages concept, working pattern, and known pitfalls into a single reusable document. By learning through skills rather than tutorials, the learner accumulates a personal library of reusable knowledge that doubles as a working second brain. The paper defines what Claude Skills are, proposes a six-type taxonomy of skills for teaching coding, shows how the types build on one another, describes the underlying learning loop, argues for the method's significance, and works a complete example end to end.

1. What Are Claude Skills?

A Claude Skill is a structured instruction document that tells Claude how to perform a specific task or use a specific tool correctly within a given environment. It is best understood as a focused, reusable knowledge artefact — a cheat sheet authored once and applied many times.

The core component is the SKILL.md file. This is a Markdown document with a small set of conventional parts:

Components of a SKILL.md file

Component	Purpose
Front matter	A name and a description that tells Claude when the skill should be triggered — the conditions under which it applies.
Overview	A short statement of what the skill is for and why it exists.
Quick reference	The minimum working pattern or command — the thing the user most often needs.
Detailed patterns	Worked examples, parameters, and variations for the task.
Common mistakes	Known pitfalls and their fixes — the hard-won, trial-and-error knowledge.
When to use instead	Pointers to alternative approaches when this skill is the wrong tool.

A skill may also bundle supporting files — scripts, reference documents, or templates — but the SKILL.md is the heart of it. The crucial property for education is this: a skill does not merely describe knowledge, it encodes the judgement of when and how to apply it. That is precisely the kind of knowledge a learner struggles to extract from conventional documentation.

The key insight

A SKILL.md has the same shape as a good coding tutorial — why, what, how, and what-not-to-do — but in a form the learner can keep, reuse, and eventually author themselves. This is what makes it a teaching instrument and not just a reference.

2. A Taxonomy of Skill Types for Teaching Coding

Different skills serve different cognitive purposes. A useful taxonomy captures that difference rather than simply listing topics. Two dimensions organise the space. The first is the level of abstraction the skill addresses, running from a single tool up to a whole architecture. The second is the role the learner takes, running from following a pattern to creating their own. Across these dimensions, six skill types cover most of the territory.

Type	Question it answers	Learner role	Example
Tool	How do I use this specific thing?	Follow → Adapt	Make a bar chart with matplotlib
Pattern	How do I solve this class of problem?	Adapt → Compose	Retry with exponential backoff
Workflow	How do I do this task end to end?	Follow → Compose	CSV to summary report
Diagnostic	How do I find what is wrong?	Follow → Adapt	Read and fix a Python error
Judgement	How do I choose between options?	Adapt → Create	List versus dictionary
Meta	How do I work better as a coder?	Create	Writing a SKILL.md

2.1 Markup and prompt skills as first-class skill types

For non-developers, two adjacent skill families deserve explicit place in the taxonomy, because for this audience they often replace conventional code entirely.

Markup skills

Markup languages — Markdown, HTML, YAML, and the front matter of a SKILL.md itself — are the gentlest possible on-ramp to thinking in structured text. They have no control flow to master, yet they teach the discipline of precise, machine-readable expression. A Markdown skill is a Tool skill in the taxonomy, but pedagogically it is the ideal first skill: the learner sees their structure render immediately, with no execution environment to configure.

Prompt skills

For a non-developer, a well-formed prompt is the program. The instruction given to Claude is the artefact that determines the output. A prompt skill therefore sits in the taxonomy as a Pattern or Workflow skill whose medium is natural language rather than Python. Treating prompts as a formal skill type — with templates, parameters, and known failure modes

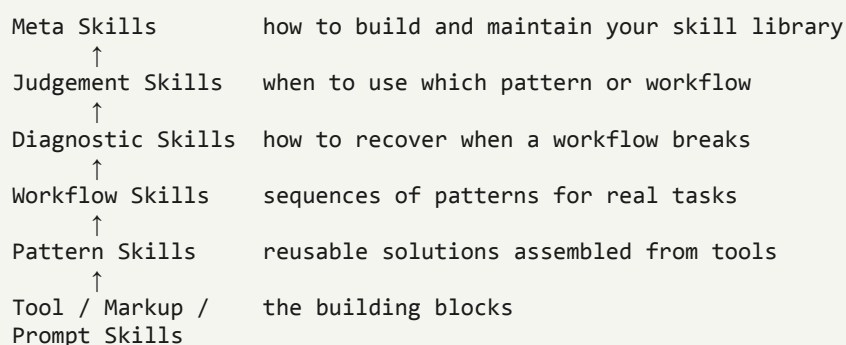
documented exactly as one would document a function — is what converts ad-hoc prompting into a transferable, improvable competency.

Why this matters for non-developers

Markup and prompt skills let a learner build real, structured, reusable artefacts before writing a single line of executable code. They lower the entry barrier without lowering the rigour — the discipline of precise specification is identical.

3. How the Skills Build on One Another

The six types are not a flat list. They form a stack, in which each layer depends on those beneath it. A well-designed curriculum moves the learner deliberately up this stack rather than simply accumulating Tool skills.

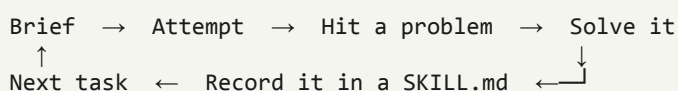


The progression has a clear logic. Tool, markup, and prompt skills supply the vocabulary. Pattern skills assemble that vocabulary into reusable solutions. Workflow skills chain patterns into complete, real-world tasks. Diagnostic skills provide the means to recover when a workflow fails — the point at which most self-taught learners stall. Judgement skills teach which workflow or pattern to reach for in the first place. Meta skills, at the summit, are skills about the skill system itself: how to capture new knowledge as a skill and how to maintain the growing library.

A learner who reaches the Meta layer has crossed from consumer to author. In the language of the Erdos curriculum, they have become a Domain Skill Designer — the primary builder of systems in a skills-based development context.

4. The Learning Loop

The method runs on a single repeating cycle. Each pass through the loop produces a tangible artefact: a new or improved skill the learner keeps.



The loop turns friction into permanent knowledge. The moment a learner hits a problem and resolves it is precisely the moment worth capturing — and the skill format gives that moment a home. The act of writing the skill is itself the consolidation step: explaining a fix in the structured form of a skill forces the learner to articulate the underlying principle, not merely the patch.

Applied across a programme, the loop has three phases:

Phase	Focus	Dominant skill types
Foundation	Read and talk about code precisely	Markup, Tool, Vocabulary
Application	Direct Claude to produce real outputs	Prompt, Workflow
Ownership	Recover from failure; author own skills	Diagnostic, Judgement, Meta

5. Significance of the Method

Several properties distinguish this approach from conventional coding instruction.

- **Active construction over passive consumption.** The learner authors the reference rather than reading someone else's. Knowledge written down in one's own words, prompted by one's own difficulty, is retained far better than knowledge merely read.
- **Knowledge persists as a reusable asset.** A tutorial is forgotten after the lesson. A skills library is a working second brain — a system the learner owns and uses in real projects, not a course they completed.
- **Specification, not syntax, is the real skill.** The method foregrounds the competency that actually transfers in an AI-assisted world: stating clearly what is wanted and verifying that it was delivered. The constraint is never Claude's capability; it is the clarity of the brief.
- **Failure is designed in, not avoided.** Diagnostic skills make debugging a first-class, teachable activity rather than the unstructured frustration where most self-taught learners abandon the effort.
- **It scales from non-coder to author.** The same loop carries a domain expert from reading their first error message to designing skill systems for their organisation — with no discontinuity in method.

6. An End-to-End Example

Consider a non-developer — a policy analyst — who wants to turn a spreadsheet of survey responses into a summary chart. They have never written code. The following traces one full pass through the method.

Step 1 — The brief

The analyst is given a single instruction: produce a bar chart of responses by region, and write a skill capturing how you did it.

Step 2 — Apply a prompt skill

Using a Prompt skill they were given on Day 1, the analyst writes a specification rather than code:

```
# Prompt specification
Input:  a CSV with columns 'region' and 'response_count'
Output: a bar chart, regions on the x-axis, counts on the y-axis
Constraint: use matplotlib only; save as chart.png
Check:  the number of bars must equal the number of regions
```

Step 3 — Claude produces the code; the analyst reads it

Claude returns a short script. Using a Tool skill on reading code, the analyst confirms each line does what the brief asked — they cannot write it from scratch, but they can verify it.

```
import matplotlib.pyplot as plt
import pandas as pd

data = pd.read_csv('responses.csv')
plt.bar(data['region'], data['response_count'], color='steelblue')
plt.title('Responses by Region')
plt.ylabel('Count')
plt.savefig('chart.png')
```

Step 4 — Hit a problem, apply a diagnostic skill

The script fails with a `KeyError`. Using a Diagnostic skill on reading errors, the analyst asks the three diagnostic questions — which line, what type, what was expected — and discovers the column is actually named 'Region' with a capital R. They correct the brief and Claude regenerates the working script.

Step 5 — Record it as a skill

Closing the loop, the analyst writes their own SKILL.md — a Meta-skill outcome built from their own domain task:

```
# SKILL: CSV to bar chart

## When to use
Comparing a single value across categories.

## The prompt that works
(the specification from Step 2)

## Common mistakes
- Column names are case-sensitive: 'Region' not 'region'
- Save the chart before showing it, or the file is blank

## When to use something else
Trend over time → line chart, not bar chart.
```

The analyst now owns a reusable asset. The next time a similar task arrives, they begin not from zero but from their own skill — and the library grows by one.

What the learner gained

Not a single chart, but a repeatable capability: the ability to specify, verify, debug, and capture. After a dozen such loops, the analyst holds a personal skills library covering the coding tasks their actual work requires — and the confidence to extend it.

7. Implementing the Method: A Teacher's Workflow

The preceding sections describe the method in principle. This section makes it operational from the educator's point of view. It sets out exactly what a teacher prepares in advance, how students reach and invoke the skills, the live teaching loop between teacher and student, and the student's own learning loop that does the real work. Together these convert the method from a concept into a deliverable classroom workflow.

The division of labour

The teacher's effort is front-loaded into preparation and concentrated at assessment. The middle of the loop — attempt, problem, diagnosis, solution — runs largely by itself, with the teacher intervening only at the seams. The student does the construction; the teacher designs the conditions for it.

7.1 What the Teacher Creates in Advance

Five artefacts are built once and reused across every cohort. Together they form a reusable teaching pack.

Artefact	What it is	Why it matters
Starter skill set	The skills students are given but do not write — typically reading-errors, code-vocabulary, prompt-spec, verify-code, and one or two domain workflow skills.	Supplies the scaffolding. Students apply these from day one before they can author their own.
Blank skill template	An empty SKILL.md with four sections: when to use, the pattern that works, common mistakes, when to use something else.	The single most important artefact — it defines the shape of everything students will author.
Task briefs	The prompts that start each loop, each ending with the instruction to write a skill capturing how it was done.	Sequence them up the taxonomy stack: markup, then prompt, then workflow, then diagnostic, then judgement.
Assessment rubric	Criteria for grading a student-authored skill — above all, whether it captures the principle behind a fix, not merely the patch.	This is where genuine understanding is assessed, not mere task completion.
Skill index	A one-page list of every starter skill with its trigger condition.	Teaches students which skill applies to a situation — a judgement skill in itself.

The blank skill template deserves particular emphasis. Because every pass through the learning loop ends by filling it in, its structure silently shapes how students learn to think

about a problem. Keeping it small and disciplined is therefore a pedagogical decision, not merely a formatting one.

7.2 How Students Invoke the Skills

For non-developers, ease of invocation matters more than sophistication. Three mechanisms are workable, listed in order of how little friction they impose on the learner.

Mechanism	How it works	Best for
Shared Project (recommended)	The teacher adds all starter skills as knowledge in a Claude Project. Students work inside the project, so skills are always in context and invocation is automatic.	Beginner cohorts; the foundation phase. Removes the burden of choosing a skill.
Upload per task	The student starts a fresh conversation and uploads the one relevant skill before stating the task.	Later phases, where you want students practising the judgement of which skill fits.
Paste at the top	The student pastes the skill text as the first message of a conversation.	One-off workshops with no setup; lowest-tech, works anywhere.

A natural progression begins the cohort in a shared Project, then moves to per-task upload once choosing the right skill becomes a learning objective in its own right. Where the Skills feature is enabled in the workspace, the starter skills may instead be installed as auto-triggering skills; the Project approach, however, works regardless of configuration and is the safer default for a class.

The invocation progression

Start in a shared Project so invocation is automatic and effort goes into the task. Move to per-task upload when you deliberately want students to practise selecting the right skill — the moment choosing a skill becomes part of the lesson rather than an obstacle to it.

7.3 The Live Teaching Loop

Per task, the teacher's role is light: set up, then step back and intervene only at the seams. The discipline is to coach the diagnosis, never to supply the answer.

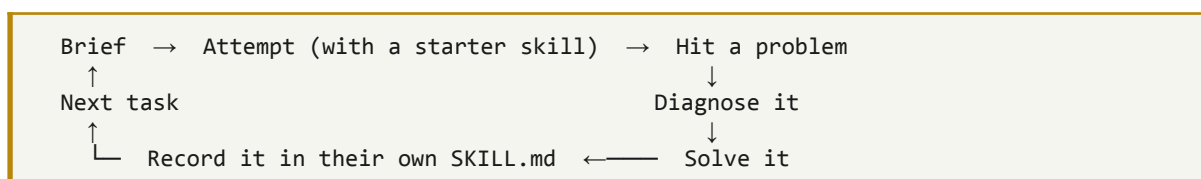
1. The teacher issues a brief from the prepared set, stating the task and the standing instruction to write a skill afterwards.
2. The student attempts it using the relevant starter skill, with Claude as the execution layer.
3. The student hits a problem. This is the point of the lesson, not a failure.
4. The teacher coaches the diagnosis, not the answer — pointing to the diagnostic skill and asking which checks have been made, letting the student find the cause.
5. The student writes their own skill in the blank template.

6. The teacher assesses it against the rubric, reading closely only here, and specifically for whether the underlying principle was captured.
7. The teacher issues the next brief, one rung further up the taxonomy stack.

The teacher's attention is therefore spent on two things only: designing good briefs in advance, and judging authored skills at the end. The remainder is the student's work.

7.4 The Student's Learning Loop

Sitting inside steps two to five above is the cycle that actually produces the learning. Each pass yields a tangible artefact — a skill the student keeps.



Consolidation happens at the moment the student records the skill. Forcing them to explain a fix in the structured form of a skill converts a one-off solution into transferable knowledge — it compels articulation of the principle, not merely the patch. After a dozen such loops the student holds a personal skills library covering the coding tasks their real work requires, owned by them rather than by the teacher.

7.5 How It Adds Up Over a Course

Mapped onto the three phases of the method, the teacher's briefs, the invocation mechanism, and the student's output evolve together across the programme.

Phase	Teacher's briefs target	Student invokes via	Student produces
Foundation	Markup, vocabulary, reading code	Shared Project (automatic)	First few skills, heavily scaffolded
Application	Prompt specification, domain workflows	Shared Project, then per-task upload	Workflow skills from real tasks
Ownership	Diagnosing failure, choosing between options	Per-task upload (student selects)	Diagnostic and judgement skills, then own templates

The decisive moment arrives when students begin writing skills the teacher never assigned — capturing problems drawn from their own work. At that point the student has crossed from learner to author, from course participant to Domain Skill Designer, and the method has achieved its purpose. The teacher's remaining role is curation: helping maintain and organise a library the student now drives.

8. Conclusion

Teaching non-developers to code is the wrong goal. The right goal is teaching them to direct, read, and verify code through Claude — and to capture what they learn in a form they can reuse. The SKILL.md file is uniquely suited to this purpose because it unites concept, pattern, and pitfall in a single artefact that the learner can both use and, ultimately, author.

The six-type taxonomy — spanning tool, markup, prompt, pattern, workflow, diagnostic, judgement, and meta skills — gives the method structure, while the learning loop gives it motion. Each pass converts a moment of friction into a permanent, reusable skill. The result is not a course that ends but a personal skills library that compounds: a working second brain for coding, owned by the learner and grounded in their own domain.

This positions the skills-as-substrate approach not as a gentler route to programming, but as a more durable one — carrying the domain expert from their first error message to the role of skill-system author, with a single, consistent method throughout.

Erdos Research — AI Research & Education

This position paper is part of the Claude for Non-Developers programme.